

Project Overview: Intelligent Library Demographics System

This application is an enterprise-grade solution designed to automate library inventory management and analyze reader demographics. It solves a real-world challenge by mapping reading habits across a complete human lifespan—**tracking reader ages ranging from toddlers at 3 years old to seniors at 110 years old.**

The architecture is built entirely in Python and satisfies the three critical stages required for advanced data systems: **Document Processing (Stage 1)**, **Intelligent Agent Core (Stage 2)**, and **Cloud-Ready Containerization (Stage 3)**. [1]

The 3 Main Stages of the Application

Stage 1: Document Selection and Processing

- **The Data Document:** The system uses `library_inventory.csv` as its primary structured document. This file acts as the library's live database, holding metadata fields such as Book Title, Author, Genre, Available Copies, Physical Corridor Location, and the Target Average Reader Age.
- **The Code Processor:** Using the pandas library, the application structurally ingests this document. It converts raw comma-separated values into memory-efficient data frames, allowing the application to calculate analytics instantly without stalling.

Stage 2: Building the Intelligent Agent (LangChain)

- **The AI Orchestrator:** The application integrates **LangChain** to act as an autonomous agent. Instead of relying on rigid keyword searches, the agent can understand natural human phrasing in Spanish (e.g., *"¿Qué leen los adultos mayores?"* or *"Busca libros de fantasía"*).
- **Tool Execution (RAG Architecture):** When a user submits a question, the LangChain agent uses a **Retrieval-Augmented Generation (RAG)** pattern. It evaluates the query, triggers a custom-built Python tool (`ConsultarArchivoBiblioteca`), extracts the relevant metrics from the dataset, and frames a natural, conversational response.

Stage 3: Cloud Deployment Readiness (OCI)

- **Containerization:** The project includes a production-grade Dockerfile. This wraps the Python scripts, the CSV dataset, and the system dependencies into a lightweight container image.
- **Oracle Cloud Infrastructure (OCI) Optimization:** The container is specifically configured to run inside an **OCI Container Instance** under

Oracle's Always Free Tier. By exposing the networking port, the application changes from a local tool (localhost) into a publicly accessible cloud link available on the open internet for evaluators.

The Interactive Dashboard Features

The front end of the application is built using **Streamlit** and **Plotly**, converting the backend data into a highly visual, interactive dashboard: [1]

1. **AI Chat Box Input:** Positioned at the top of the interface, allowing users to converse directly with the library document in real time.
2. **Demographic Scaling Graph (Bar Chart):** A dynamic visualization that maps the average reader age for each book. It displays a perfect demographic gradient, color-coded from early childhood development stories (Age 3) up to historical and philosophical volumes read by centenarians (Age 110).
3. **Physical Stock Allocation (Donut Chart):** Monitors the distribution of physical book volumes across different corridors, warning library managers which sections are reaching capacity.
4. **Data Grid & Report Downloader:** An expandable matrix at the bottom of the page that allows users to export the current status into a clean Microsoft Excel report (.xlsx) with a single click.